INGENIERÍA EN SOFTWARE

Clean Architecture

Bloque 03 · Apunte completo de estudio

Capas · Puertos y adaptadores · Patrones aplicados
Refactoring · Testing · Deploy

Instituto Técnico Superior Córdoba

Ingeniería en Software · Prof. Mateo Alexis Vranich Prieto · Córdoba 2026

#	Sección
01	Qué es Clean Architecture · La regla de dependencia
02	Las capas y sus responsabilidades
03	Puertos y adaptadores
04	Refactoring guiado
05	Dónde encajan los patrones
06	Testing en Clean Architecture

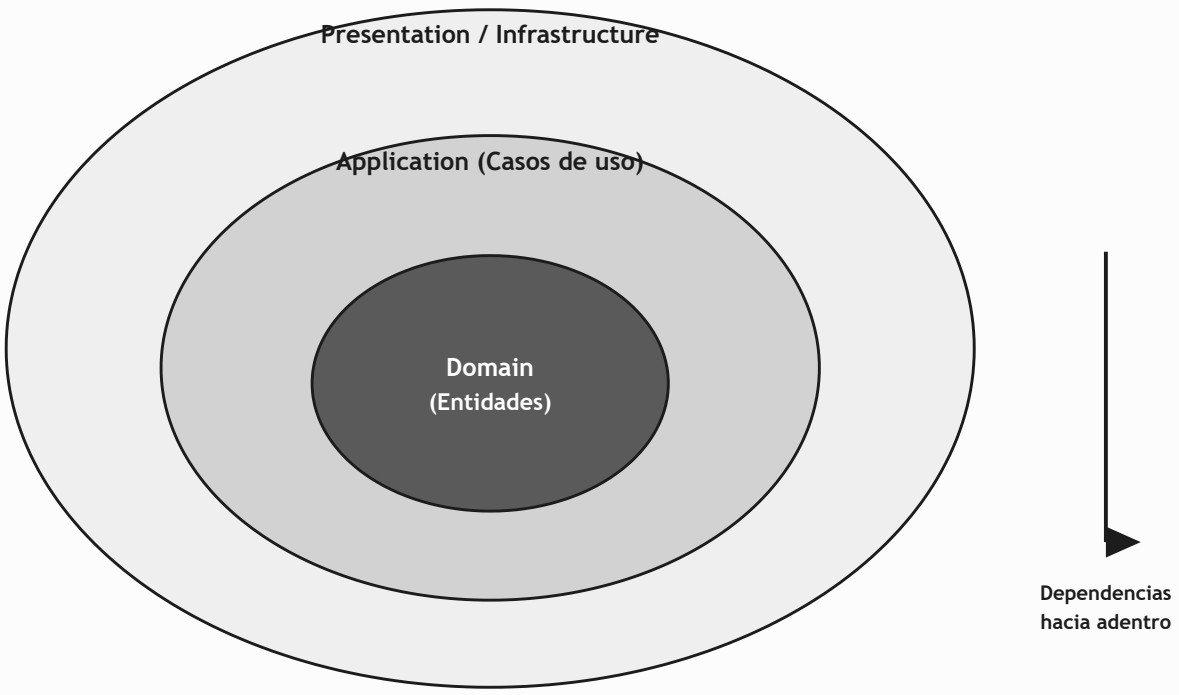
01 · ¿Qué es Clean Architecture?

Clean Architecture es un conjunto de principios de organización de software propuesto por Robert C. Martin (Uncle Bob) en 2012. No es un framework ni una librería: es una forma de pensar cómo se estructura un sistema para que sea **testeable, independiente de frameworks, independiente de la UI, independiente de la base de datos** e independiente de cualquier agente externo.

Pensá en una lomitería: el corazón del negocio es la receta del lomito, cómo se calculan los precios, las reglas de stock. Eso no cambia porque pases de mostrador a delivery, de anotar en papel a usar un sistema, de cobrar en efectivo a cobrar con QR. Clean Architecture protege ese corazón.

La regla de dependencia

Es la regla más importante y la única que no se puede romper: **las dependencias del código fuente solo pueden apuntar hacia adentro.** Las capas externas conocen a las internas, pero las internas **nunca** saben que las externas existen.



Las capas externas conocen a las internas; el Domain no conoce a nadie.

Esto significa que tu Domain (entidades y reglas de negocio) no importa ningún namespace de Entity Framework, de ASP.NET, de Angular ni de ninguna tecnología. Si querés cambiar de SQL Server a PostgreSQL, o de REST a GraphQL, el dominio no se entera.

> **Tip:** La regla de dependencia es la aplicación directa del principio D de SOLID (Inversión de Dependencias) a nivel de arquitectura. En SOLID lo aplicás clase por clase; en Clean Architecture lo aplicás capa por capa.

Conexión con lo que ya vieron

En Programación III ya trabajan con una estructura de capas: Controller → Service → Repository. Esa es una versión simplificada de Clean Architecture. La diferencia clave es que en la estructura plana de Prog III, el Service suele conocer directamente al Repository concreto (SqlRepo). En Clean Architecture, el Service define una interfaz (puerto) y la capa de Infrastructure provee la implementación (adaptador).



02 · Las capas y sus responsabilidades

Clean Architecture organiza el código en cuatro capas concéntricas. Cada capa tiene una responsabilidad clara y bien definida:

Domain (Entidades)

Es el núcleo: contiene las **entidades de negocio**, los **value objects** y las **reglas de negocio puras**. No tiene ninguna dependencia externa. No conoce la base de datos, no conoce HTTP, no conoce frameworks.

- **Entidades:** clases con identidad (Producto, Pedido, Cliente).
- **Value Objects:** objetos sin identidad definidos por sus valores (Dirección, Precio, Email).
- **Interfaces de repositorio:** los contratos (puertos) que define el dominio.
- **Excepciones de dominio:** errores específicos del negocio (StockInsuficienteException).

Domain/Entities/Producto.cs

```
public class Producto {
    public int Id { get; set; }
    public string Nombre { get; set; }
    public decimal Precio { get; set; }
    public int Stock { get; set; }

    public bool TieneStock(int cantidad)
        => Stock >= cantidad;

    public void DescontarStock(int cantidad) {
        if (!TieneStock(cantidad))
            throw new StockInsuficienteException(Nombre);
        Stock -= cantidad;
    }
}
```

Application (Casos de uso)

Contiene la **lógica de aplicación**: los casos de uso que orquestan las entidades. Conoce al Domain pero **no** conoce la infraestructura. Define interfaces para todo lo que necesita del exterior (repositorios, servicios de email, etc.).

- **Servicios de aplicación (casos de uso):** CrearPedidoUseCase, ObtenerProductosUseCase.
- **DTOs:** objetos para transferir datos entre capas sin exponer las entidades.
- **Interfaces de servicios externos:** IEmailService, INotificacionService.
- **Validaciones de aplicación:** reglas que no son de negocio puro (ej: formato de request).

Application/UseCases/CrearPedidoUseCase.cs

```
public class CrearPedidoUseCase {
    private readonly IPedidoRepo _pedidoRepo;
    private readonly IProductoRepo _productoRepo;

    public CrearPedidoUseCase(IPedidoRepo pedidos,
                              IProductoRepo productos) {
        _pedidoRepo = pedidos;
        _productoRepo = productos;
    }

    public async Task<PedidoDto> Ejecutar(CrearPedidoDto dto) {
        var producto = await _productoRepo
            .ObtenerPorIdAsync(dto.ProductoId);
        producto.DescontarStock(dto.Cantidad);
        var pedido = new Pedido(producto, dto.Cantidad);
        await _pedidoRepo.GuardarAsync(pedido);
        return PedidoDto.Desde(pedido);
    }
}
```

Infrastructure (Infraestructura)

Implementa todo lo que el Domain y Application definen como interfaces. Es la capa que **toca el mundo real**: la base de datos, las APIs externas, el sistema de archivos, el envío de emails.

- **Repositorios concretos:** SqlProductoRepo, implementan IProductoRepo.
- **DbContext** de Entity Framework Core.
- **Servicios externos:** SmtpEmailService implementa IEmailService.
- **Configuración de DI:** registro de servicios en el contenedor.

Infrastructure/Repositories/SqlProductoRepo.cs

```
public class SqlProductoRepo : IProductoRepo {
    private readonly AppDbContext _context;

    public SqlProductoRepo(AppDbContext context)
        => _context = context;

    public async Task<Producto> ObtenerPorIdAsync(int id)
        => await _context.Productos
            .FirstOrDefaultAsync(p => p.Id == id);

    public async Task GuardarAsync(Pedido pedido) {
        await _context.Pedidos.AddAsync(pedido);
        await _context.SaveChangesAsync();
    }
}
```

Presentation (Presentación / API)

Es el punto de entrada: controladores HTTP, Program.cs, configuración de middleware. Conoce a Application (para llamar los casos de uso) y a Infrastructure (para registrar las implementaciones en DI). No contiene lógica de negocio.

Presentation/Controllers/PedidosController.cs

```
[ApiController]
[Route("api/[controller]")]
public class PedidosController : ControllerBase {
    private readonly CrearPedidoUseCase _crearPedido;

    public PedidosController(CrearPedidoUseCase uc)
        => _crearPedido = uc;

    [HttpPost]
    public async Task<IActionResult> Crear(
        [FromBody] CrearPedidoDto dto) {
        var result = await _crearPedido.Ejecutar(dto);
        return CreatedAtAction(..., result);
    }
}
```

Estructura de carpetas

Proyecto Clean Architecture en .NET

```
MiLomiteria.sln
├─ MiLomiteria.Domain/          Entities/  Interfaces/  Exceptions/
├─ MiLomiteria.Application/     UseCases/  DTOs/  Interfaces/
├─ MiLomiteria.Infrastructure/  Repositories/  Data/  Services/
└─ MiLomiteria.API/             Controllers/  Program.cs
```

> **Tip:** En .NET, cada capa suele ser un **proyecto separado** dentro de la misma solución (.sln). Así el compilador te obliga a respetar la regla de dependencia: si Domain no tiene referencia a Infrastructure, no podés importar Entity Framework desde una entidad.

03 · Puertos y adaptadores

La arquitectura hexagonal (Ports & Adapters), propuesta por Alistair Cockburn, es la base conceptual de Clean Architecture. La idea es simple: el núcleo del sistema define **puertos** (interfaces) y el exterior provee **adaptadores** (implementaciones).

Concepto	En código	Ejemplo lomitería
Puerto (interfaz)	`IProductoRepo`	El mostrador: define cómo se pide
Adaptador (impl.)	`SqlProductoRepo`	El cocinero específico que cocina hoy
Puerto de salida	`IEmailService`	El teléfono del delivery
Adaptador de salida	`SmtpEmailService`	El delivery que contrataste

Lo clave: **el puerto lo define la capa interna** (Domain o Application) y **el adaptador lo implementa la capa externa** (Infrastructure). Así, el núcleo nunca depende de la tecnología concreta.

Lo que ya hacés en Programación III es esto

Puerto + Adaptador + Registro

```
// PUERTO (lo define Application)
public interface IProductoRepo {
    Task<List<Producto>> ObtenerTodosAsync();
    Task<Producto> ObtenerPorIdAsync(int id);
    Task GuardarAsync(Producto p);
}

// ADAPTADOR (lo implementa Infrastructure)
public class SqlProductoRepo : IProductoRepo {
    private readonly AppDbContext _ctx;
    // ... implementa los métodos usando EF Core
}

// REGISTRO en Program.cs (la conexión)
builder.Services.AddScoped<IProductoRepo, SqlProductoRepo>();
```

> **Tip:** Si mañana querés reemplazar SQL Server por MongoDB, solo creás un MongoProductoRepo que implemente la misma interfaz y cambiás UNA línea en Program.cs. Cero cambios en el dominio, cero cambios en los controllers. Eso es el poder de los puertos y adaptadores.

Tipos de puertos

- **Puertos de entrada (driving):** definen cómo el exterior interactúa con el sistema. Ejemplo: un caso de uso (CrearPedidoUseCase) es un puerto de entrada — el controller lo llama.
- **Puertos de salida (driven):** definen cómo el sistema interactúa con el exterior. Ejemplo: IProductoRepo, IEmailService — el caso de uso los usa, pero no sabe cómo funcionan por dentro.

04 · Refactoring guiado

Veamos cómo transformar un proyecto típico de Programación III (estructura plana) en Clean Architecture, paso a paso. Usamos el ejemplo de "La Lomi App", el sistema de gestión de la lomitería.

ANTES: estructura plana

LomiApp/ (estructura Prog III)

```
LomiApp/
├─ Controllers/     ProductosController.cs
├─ Services/        ProductoService.cs
├─ Repositories/    SqlProductoRepo.cs
├─ Models/          Producto.cs
├─ Data/            AppDbContext.cs
└─ Program.cs
```

El problema: todo vive en el mismo proyecto. El Service importa directamente EF Core. El Model puede tener Data Annotations de la BD mezcladas con lógica de negocio. Si querés testear el Service, necesitás la base de datos real.

DESPUÉS: Clean Architecture

LomiApp.sln (Clean Architecture)

```
LomiApp.Domain/          Entities/Producto.cs  Interfaces/IProductoRepo.cs
LomiApp.Application/     UseCases/ObtenerProductosUseCase.cs  DTOs/ProductoDto.cs
LomiApp.Infrastructure/  Repositories/SqlProductoRepo.cs  Data/AppDbContext.cs
LomiApp.API/             Controllers/ProductosController.cs  Program.cs
```

Pasos del refactoring

- **Paso 1:** Extraer las entidades al proyecto Domain. Quitar toda referencia a EF Core (sin [Table], sin [Key], sin [Required] de System.ComponentModel). Las Data Annotations de validación se mueven a DTOs o a la configuración de EF (Fluent API).
- **Paso 2:** Mover las interfaces (IProductoRepo, IProductoService) al proyecto Domain o Application, según corresponda.
- **Paso 3:** Crear el proyecto Application con los casos de uso. El Service de Prog III se convierte en un UseCase que recibe interfaces por constructor.
- **Paso 4:** Mover los repositorios concretos y el DbContext al proyecto Infrastructure.
- **Paso 5:** Configurar las referencias entre proyectos respetando la regla:

Proyecto	Puede referenciar a	NO puede referenciar a
Domain	Nada (es el núcleo)	Application, Infrastructure, API
Application	Domain	Infrastructure, API
Infrastructure	Domain, Application	API
API	Application, Infrastructure	(solo para DI registration)

05 · Dónde encajan los patrones de diseño

Los patrones de diseño que vieron en el primer período no son teoría suelta: cada uno tiene un lugar natural en Clean Architecture. Veamos dónde vive cada uno:

Patrón	Capa	Para qué se usa	Ejemplo en la lomitería
Repository	Domain (interfaz) + Infrastructure (impl.)	Abstraer el acceso a datos detrás de un contrato	IProductoRepo define los métodos; SqlProductoRepo los implementa con EF Core
Factory Method	Application	Crear objetos complejos sin exponer la lógica de creación	NotificacionFactory crea EmailNotif o PushNotif según configuración
Strategy	Application	Intercambiar algoritmos en runtime sin cambiar el contexto	IDescuento con DescuentoVIP y DescuentoEstudiante; el UseCase elige cuál aplicar
Decorator	Infrastructure	Añadir comportamiento (logging, caché) sin modificar la clase original	CachedProductoRepo envuelve a SqlProductoRepo y cachea resultados
Observer	Application / Infrastructure	Notificar a múltiples suscriptores cuando algo cambia	Cuando se crea un pedido, se notifica a Cocina, Caja y Stock
Adapter	Infrastructure	Adaptar una API externa a la interfaz que espera el dominio	MercadoPagoAdapter implementa IPagoService adaptando la SDK de MP

Ejemplo: Decorator para caché

El patrón Decorator permite agregar caché al repositorio sin tocar el código original. El CachedProductoRepo implementa la misma interfaz y envuelve al repositorio real:

Infrastructure/Repositories/CachedProductoRepo.cs

```
public class CachedProductoRepo : IProductoRepo {
    private readonly IProductoRepo _inner;
    private readonly IMemoryCache _cache;

    public CachedProductoRepo(IProductoRepo inner,
                              IMemoryCache cache) {
        _inner = inner;
        _cache = cache;
    }

    public async Task<Producto> ObtenerPorIdAsync(int id) {
        string key = $"producto:{id}";
        if (_cache.TryGetValue(key, out Producto p))
            return p;
        p = await _inner.ObtenerPorIdAsync(id);
        _cache.Set(key, p, TimeSpan.FromMinutes(5));
        return p;
    }
}
```

En Program.cs, registrás primero el repo real y después lo envolvés con el decorador. Ningún otro código se entera de que hay caché — solo ve IProductoRepo.

> **Conexión con SOLID:** El Decorator funciona gracias a los principios O (agregás caché sin modificar el repo original), L (el decorator es sustituible por el repo real), y D (ambos dependen de la abstracción IProductoRepo).

06 · Testing en Clean Architecture

Una de las ventajas más grandes de Clean Architecture es que hace el testing mucho más fácil. ¿Por qué? Porque cada capa depende de interfaces, y las interfaces se pueden **mockear** (reemplazar con implementaciones falsas para testing).

Qué se testea en cada capa

Capa	Tipo de test	Qué se testea	Se mockea
Domain	Unitario	Lógica de entidades y value objects	Nada (no tiene dependencias)
Application	Unitario	Casos de uso (orquestación)	Repositorios e interfaces externas
Infrastructure	Integración	Que el repo realmente persista en la BD	Nada (BD real o in-memory)
API	Integración / E2E	Que los endpoints respondan correctamente	Se usa WebApplicationFactory

Ejemplo: test unitario de un caso de uso

Como CrearPedidoUseCase recibe IProductoRepo y IPedidoRepo por constructor, podemos inyectar mocks y testear la lógica sin base de datos:

Tests/Application/CrearPedidoUseCaseTests.cs

```
[Fact]
public async Task Crear_ConStock_DescuentaYGuarda() {
    // Arrange
    var producto = new Producto {
        Id = 1, Nombre = "Lomito especial",
        Precio = 5500, Stock = 10
    };
    var mockProductoRepo = new Mock<IProductoRepo>();
    mockProductoRepo.Setup(r => r.ObtenerPorIdAsync(1))
        .ReturnsAsync(producto);
    var mockPedidoRepo = new Mock<IPedidoRepo>();

    var useCase = new CrearPedidoUseCase(
        mockPedidoRepo.Object, mockProductoRepo.Object);

    // Act
    var dto = new CrearPedidoDto { ProductoId = 1, Cantidad = 2 };
    var result = await useCase.Ejecutar(dto);

    // Assert
    Assert.Equal(8, producto.Stock);
    mockPedidoRepo.Verify(
        r => r.GuardarAsync(It.IsAny<Pedido>()), Times.Once);
}
```

> **Conexión con Bloque 02:** Esto es la pirámide de testing en acción. Los tests unitarios del Domain y Application son la base (rápidos, muchos). Los tests de integración de Infrastructure están en el medio. Los E2E de la API están en la cima (pocos, lentos).

¿Por qué sin Clean Architecture es más difícil?

Si el Service usa directamente **new SqlProductoRepo()** en lugar de recibir una interfaz, no podés inyectar un mock. Necesitás una base de datos real para correr los tests. Eso hace que los tests sean lentos, frágiles y difíciles de mantener. Clean Architecture + DI + interfaces = tests rápidos y confiables.

07 · Del commit al deploy

Ya vieron Docker y GitHub Actions. Ahora veamos cómo Clean Architecture se empaqueta y despliega sin repetir lo que ya dieron, solo conectando los conceptos.

Estructura de proyectos en el Dockerfile

Como Clean Architecture tiene múltiples proyectos (.csproj), el Dockerfile necesita copiar todos antes de hacer el build:

Dockerfile

```
# Dockerfile para Clean Architecture .NET
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src

# Copiar todos los .csproj primero (cache de restore)
COPY MiLomiteria.Domain/*.csproj Domain/
COPY MiLomiteria.Application/*.csproj Application/
COPY MiLomiteria.Infrastructure/*.csproj Infra/
COPY MiLomiteria.API/*.csproj API/
RUN dotnet restore API/MiLomiteria.API.csproj

# Copiar todo el código y compilar
COPY . .
RUN dotnet publish API/MiLomiteria.API.csproj -c Release -o /app

FROM mcr.microsoft.com/dotnet/aspnet:8.0
WORKDIR /app
COPY --from=build /app .
ENTRYPOINT ["dotnet", "MiLomiteria.API.dll"]
```

Pipeline de GitHub Actions

El pipeline para un proyecto Clean Architecture sigue el mismo patrón que ya conocen, pero aprovecha la separación de capas para correr tests por proyecto:

.github/workflows/ci.yml

```yaml
name: CI
on: [push, pull_request]
jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-dotnet@v4
        with: { dotnet-version: '8.0.x' }

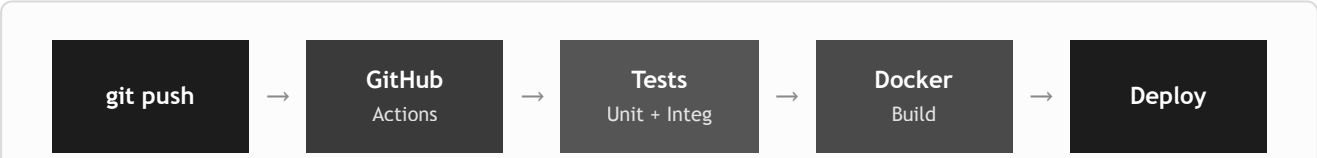
      - run: dotnet restore
      - run: dotnet build --no-restore

      # Tests unitarios (rápidos, sin BD)
      - run: dotnet test Tests/Domain.Tests
      - run: dotnet test Tests/Application.Tests

      # Tests de integración (con BD)
      - run: dotnet test Tests/Infrastructure.Tests

      # Build de la imagen Docker
      - run: docker build -t milomiteria .
```

El flujo completo



> **Tip:** La separación en capas tiene un beneficio extra en CI: si solo cambiaste código en Domain, los tests unitarios del Domain fallarán rápido sin esperar a los de integración. Feedback más rápido = menos tiempo perdido.

Resumen del bloque

Concepto	Idea clave	Conexión
Regla de dependencia	Las dependencias apuntan hacia adentro	Principio D de SOLID
Domain	Entidades y reglas de negocio puras	Models de Prog III, sin EF Core
Application	Casos de uso que orquestan el dominio	Services de Prog III
Infrastructure	Implementaciones concretas (BD, email)	Repositories de Prog III

Concepto	Idea clave	Conexión
Presentation	Controllers y configuración HTTP	Controllers de Prog III
Puertos	Interfaces definidas por las capas internas	Principio D + I de SOLID
Adaptadores	Implementaciones que conectan con el exterior	Patrón Adapter
Testing	Mockear puertos para tests rápidos	Pirámide de Bloque 02

Glosario del bloque

Término	Definición
Clean Architecture	Conjunto de principios para organizar software en capas concéntricas con dependencias hacia adentro.
Regla de dependencia	Las capas internas nunca conocen a las externas. Las dependencias del código apuntan hacia el centro.
Puerto	Interfaz definida por una capa interna que especifica qué necesita del exterior sin saber cómo.
Adaptador	Implementación concreta de un puerto. Vive en la capa externa (Infrastructure).
Caso de uso	Operación específica de la aplicación que orquesta entidades del dominio.
DTO	Data Transfer Object. Objeto para transferir datos entre capas sin exponer entidades internas.
Value Object	Objeto definido por sus valores, sin identidad propia (ej: Dirección, Precio).
Mock	Implementación falsa de una interfaz usada para testing. Simula el comportamiento real.